

DATABASE MANAGEMENT SYSTEMS

Syllabus: Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

UNIT 1 PART 2

Entity Relation Model

The Entity Relationship (ER) data model allows us to describe the data involved in a real-world enterprise in terms of objects and their relationships and is widely used to develop an initial data base design. Within the larger context of the overall design process, the ER model is used in a phase called. “**Conceptual database design**”.

ER Modeling helps you to analyze data requirements systematically to produce a well-designed database. So, it is considered a best practice to complete ER modeling before implementing your database.

ER diagrams are a visual tool which is helpful to represent the ER model. It was proposed by **Peter Chen** in 1971 to create a uniform convention which can be used for relational database and network.

Database design and ER Diagrams:

The database design process can be divided into six steps. The ER model is most relevant to the first three steps.

1. Requirement Analysis:

The very first step in designing a database application is to understand what data is to be stored in the database, what application must be built in top of it, and what operations are most frequent and subject to performance requirements. In other words, we must find out what the users want from the database.

2. Conceptual database Design:

The information gathered in the requirements analysis step is used to develop a high-level description of the data to be stored in the database, along with the constraints

DATABASE MANAGEMENT SYSTEMS

known to hold over this data. The ER model is one of several high level or semantic, data models used in database design.

3. Logical Database Design:

We must choose a database to convert the conceptual database design into a database schema in the data model of the chosen DBMS. Normally we will consider the Relational DBMS and therefore, the task in the logical design step is to convert an ER schema into a relational database schema.

Beyond ER Design

4. Schema Refinement:

This step is to analyze the collection of relations in our relational database schema to identify potential problems, and refine it.

5. Physical Database Design:

This step may simply involve building indexes on some table and clustering some tables or it may involve substantial redesign of parts of database schema obtained from the earlier steps.

6. Application and security Design:

Any software project that involves a DBMS must consider aspects of the application that go beyond the database itself. We must describe the role of each entity (users, user group, departments) in every process that is reflected in some application tasks, as part of a complete workflow for the task. A DBMS Provides several mechanisms to assist in this step.

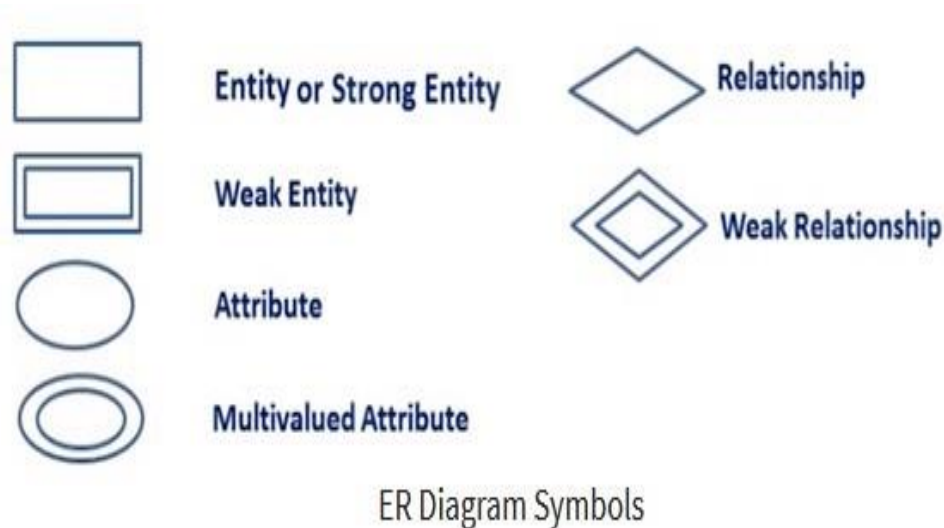
ER Diagrams Symbols & Notations

Entity Relationship Diagram Symbols & Notations mainly contains three basic symbols which are **rectangle**, **oval** and **diamond** to represent relationships between elements, entities, and attributes. There are some sub-elements which are based on main elements in ERD Diagram. ER Diagram is a visual representation of data that describes how data is related to each other using different ERD Symbols and Notations.

DATABASE MANAGEMENT SYSTEMS

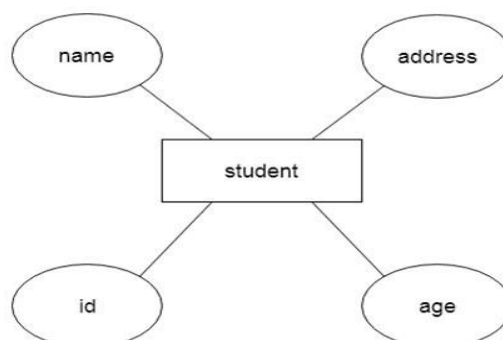
Following are the main components and its symbols in ER Diagrams:

- **Rectangles:** This Entity Relationship Diagram symbol represents entity types
- **Ellipses:** Symbol represent attributes
- **Diamonds:** This symbol represents relationship types
- **Lines:** It links attributes to entity types and entity types with other relationship types
- **Primary key:** attributes are underlined.
- **Double Ellipses:** Represent multi-valued attributes.



[ER Diagram Examples](#)

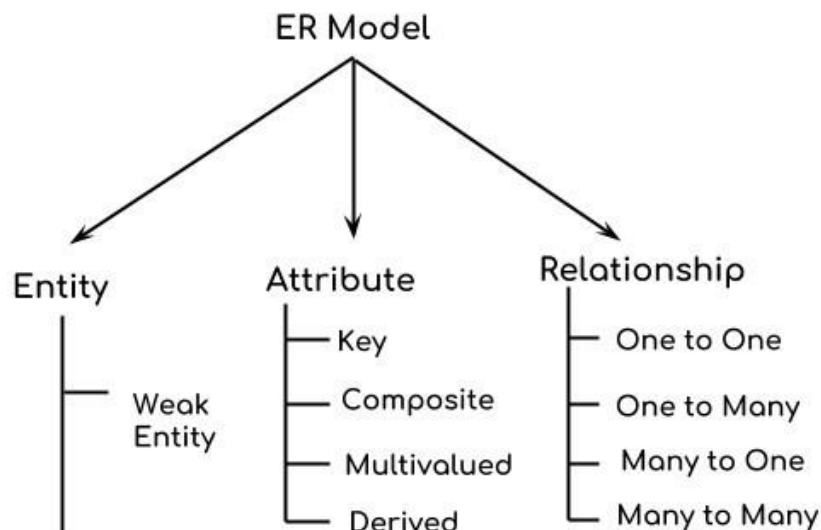
Suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc. The address can be another entity with attributes like city, street name, pin code, etc. and there will be a relationship between them.



Components of the ER Diagram

This model is based on three basic concepts:

- **Entities**
- **Attributes**
- **Relationships**



Representation of entities

An entity can be place, person, object, event, or a concept, which stores data in the database. The characteristics of entities must have an attribute, and a unique key. Every entity is made up of some 'attributes' which represent that entity.

An entity is an object or component of data. An entity is represented as **rectangle** in an ER diagram.

Examples of entities:

- **Person:** Employee, Student, Patient.
- **Place:** Store, Building.
- **Object:** Machine, product, and Car.
- **Event:** Sale, Registration, Renewal.
- **Concept:** Account, Course.

DATABASE MANAGEMENT SYSTEMS

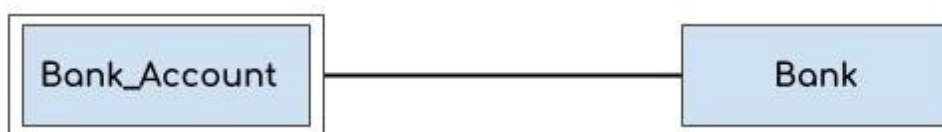
Forexample: In the following ER diagram we have two entities Student and College, and these two entities have many to one relationship as many student's study in a single college.



Weak Entity

An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is represented by a **double rectangle**.

For example - a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



Attributes

An attribute describes the property of an entity. For example, Roll_No, Name, DOB, Age, Address, Mobile_No are the attributes which defines entity type Student. In ER diagram, attribute is represented by an **oval**.



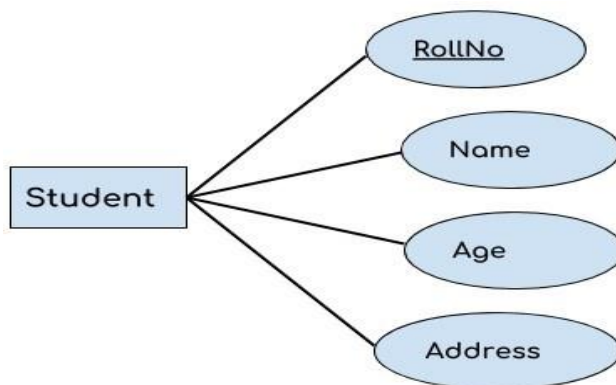
There are four types of attributes:

1. Key attribute
2. Composite attribute
3. Multivalued attribute
4. Derived attribute

DATABASE MANAGEMENT SYSTEMS

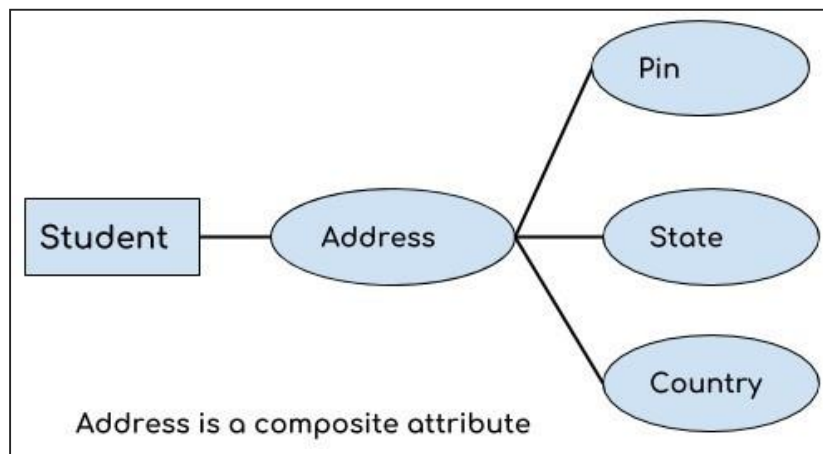
1. Key attribute

A key attribute can uniquely identify an entity from an entity set. For example, student roll number can uniquely identify a student from a set of students. Key attribute is represented by oval same as other attributes however the **text of key attribute is underlined**.



2. Composite attribute

An attribute that is a combination of other attributes is known as composite attribute. For example, in student entity, the student address is a composite attribute as an address is composed of other attributes such as pin code, state, country.



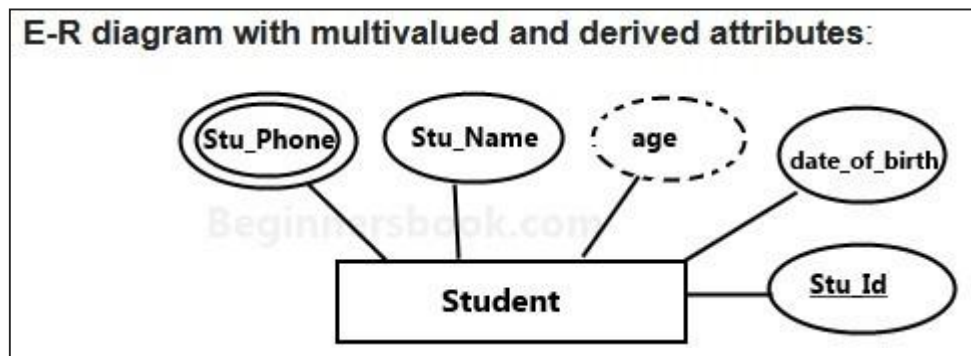
3. Multivalued attribute

An attribute that can hold multiple values is known as multivalued attribute. It is represented with **double ovals** in an ER Diagram. For example - A person can have more than one phone numbers so the phone number attribute is multivalued.

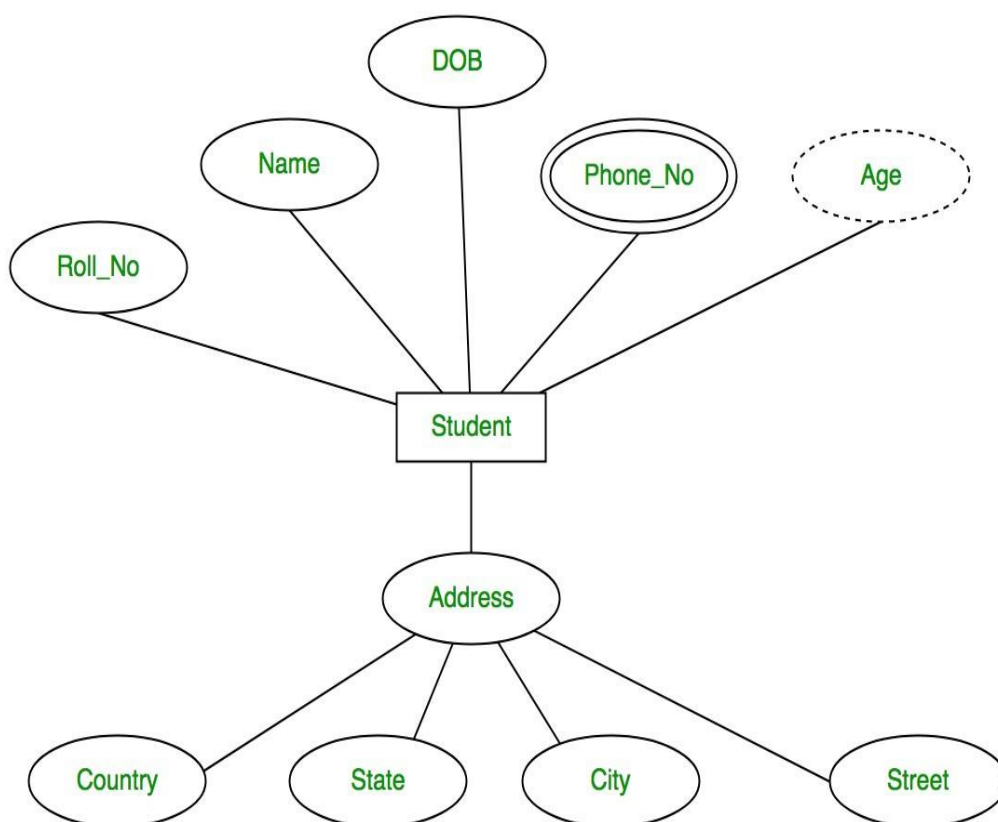
DATABASE MANAGEMENT SYSTEMS

4. Derived attribute:

A derived attribute is one whose value is dynamic and derived from another attribute. It is represented by **dashed oval** in an ER Diagram. For example - Person age is a derived attribute as it changes over time and can be derived from another attribute (Date of birth).



The complete entity type **Student** with its attributes can be represented as:



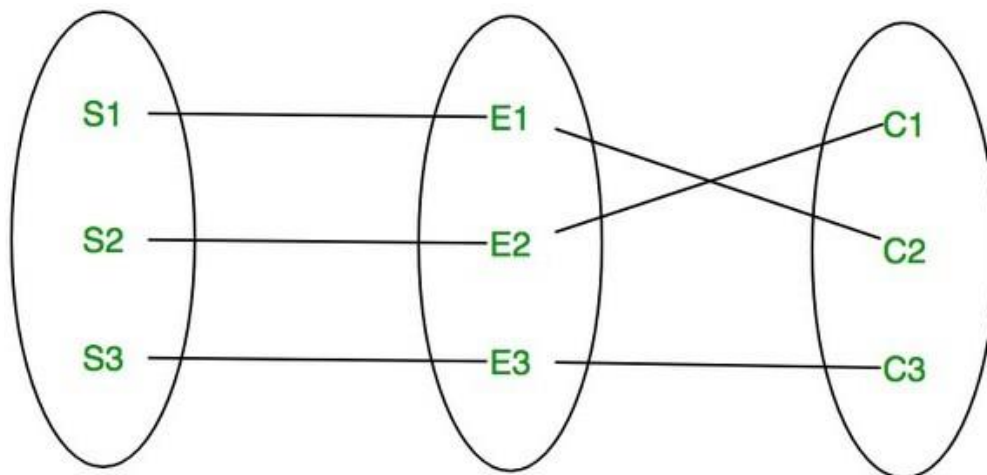
Relationship Type and Relationship Set

A relationship type represents the association **between entity types**. For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course.

A relationship is represented by diamond shape and connecting the entities with lines in ER diagram, Name of the relationship is written inside the diamond-box, it shows the relationship among entities.



A set of relationships of same type is known as **relationship set**. The following relationship set depicts S1 is enrolled in C2, S2 is enrolled in C1 and S3 is enrolled in C3.



There are four types of relationships:

1. One to One
2. One to Many
3. Many to One
4. Many to Many

DATABASE MANAGEMENT SYSTEMS

1. One to One Relationship

When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. **For example**, if there are two entities 'Person' (Id, Name, Age, Address) and 'Passport' (Passport_id, Passport_no). So, each person can have only one passport and each passport belongs to only one person.

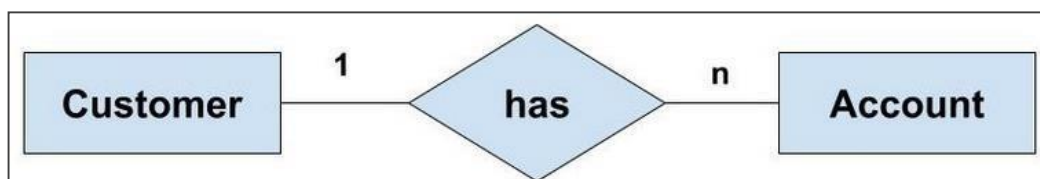


In the above example, we can easily store the passport id in the 'Person' table only. But we make another table for the 'Passport' because Passport number may be sensitive data and it should be hidden from certain users. So, by making a separate table we provide extra security that only certain database users can see it.

2. One to Many Relationship

When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationship.

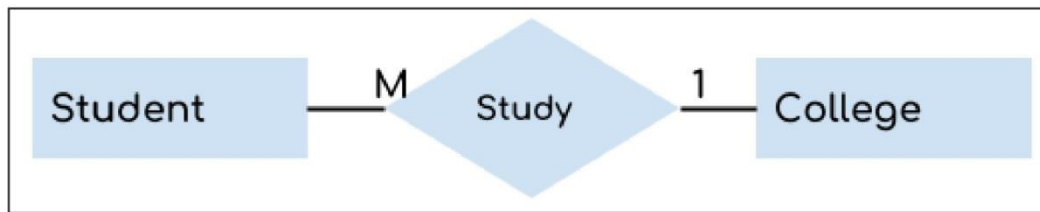
For example, if there are two entity type 'Customer' and 'Account' then each 'Customer' can have more than one 'Account' but each 'Account' is held by only one 'Customer'.



In this example, we can say that each Customer is associated with many Account. So, it is a one-to-many relationship. But if we see it the other way i.e., many Account is associated with one Customer then we can say that it is a many-to-one relationship.

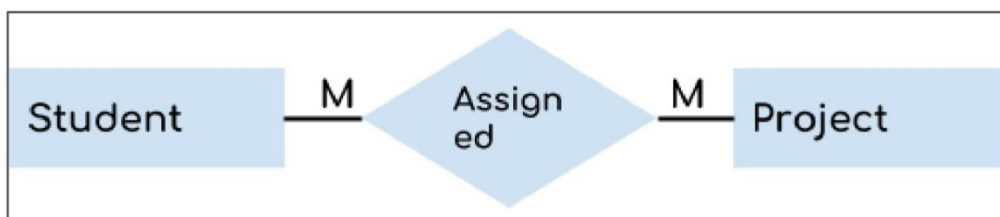
3. Many to One Relationship: When more than one instances of an entity is associated with a single instance of another entity then it is called many to one

relationship. For example - many students can study in a single college, but a student cannot study in many colleges at the same time.



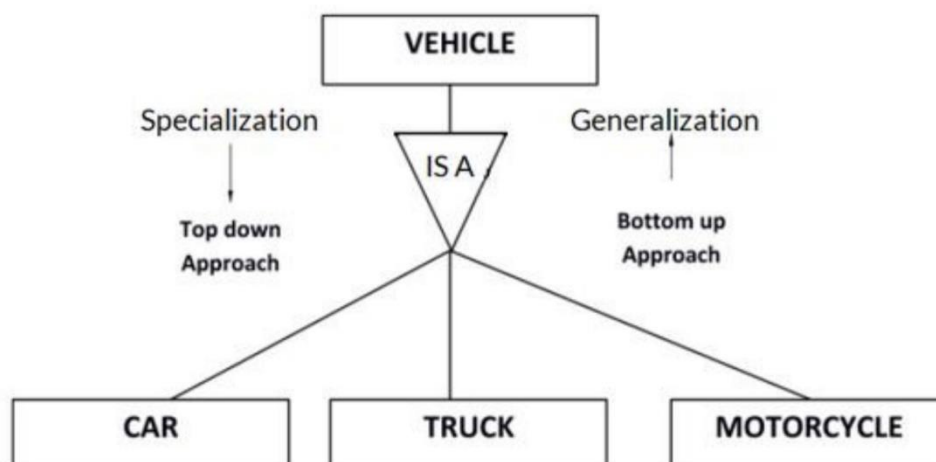
4. Many to Many Relationship

When more than one instances of an entity is associated with more than one instances of another entity then it is called many to many relationship. For example, a student can be assigned to many projects and a project can be assigned to many students.



Sub classes, Super classes, Inheritance

A subclass is a class derived from the superclass. It inherits the properties of the superclass and also contains attributes of its own.



In the above Example Car, Truck and Motorcycle are all subclasses of the superclass Vehicle. They all inherit common attributes from vehicle such as speed, color

DATABASE MANAGEMENT SYSTEMS

etc. while they have different attributes also i.e., Number of wheels in Car is 4 while in Motorcycle is 2.

Super classes

A superclass is the class from which many subclasses can be created. The subclasses inherit the characteristics of a superclass. The superclass is also known as the parent class or base class.

In the above example, Vehicle is the Superclass and its subclasses are Car, Truck and Motorcycle.

Inheritance

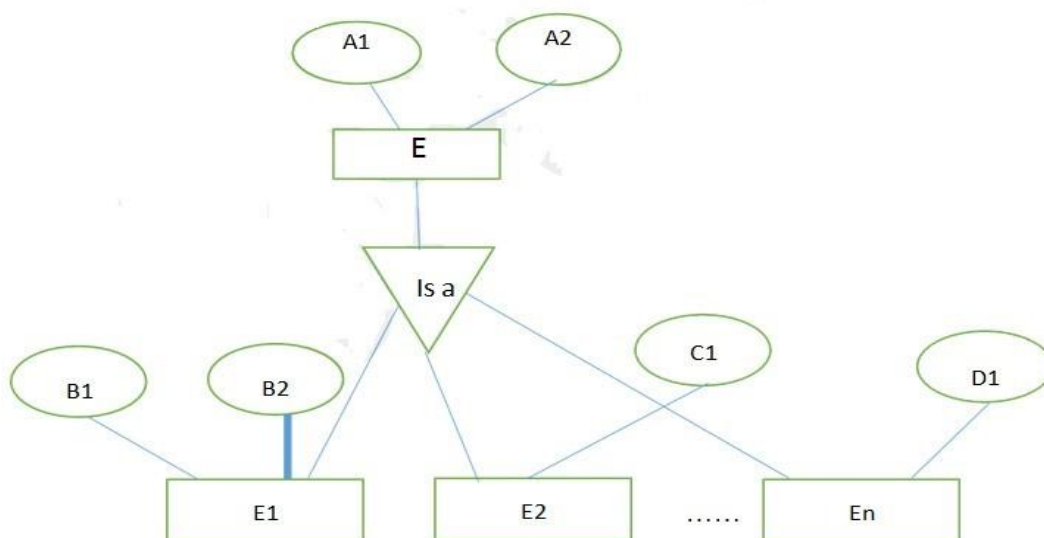
Inheritance is basically the process of basing a class on another class i.e., to build a class on an existing class. The new class contains all the features and functionalities of the old class in addition to its own.

The class which is newly created is known as the subclass or child class and the original class is the parent class or the superclass.

Specialization

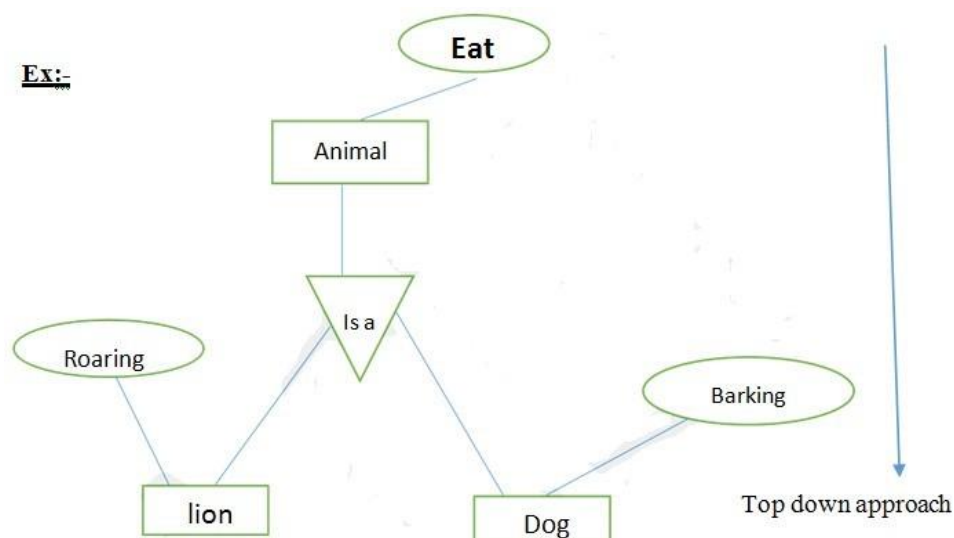
Specialization is the process of designating the sub-entities of an entity set depending upon the commonality of attributes is called specialization. An entity set **E** may include some sub-groups of entities say (**E1, E2,..... , En**) such that each of these may have some distinct or specific attributes that the other sub-groups or sub-entities. There will be some attributes common to all sub groups.

E-R model:-



- In the above example entity set **E** has been specialized into sub entity sets designated as **E1, E2,....., En**.
- **E** is called “**super class**” or “**higher level entity set**” and the entity sets **E1, E2, - - En** are called as “**sub class**” or “**low level entity sets**” of **E**. the common attribute of all sub entity sets are represented with super entity sets.
- The specific attributes of each sub entity set are represented with the sub entity set.
- The relationship of higher level entity set with the lower level entity set is called ‘is a’ relationship. i.e. **E1** is a sub group or sub entity of **E**. etc.....

Ex:-

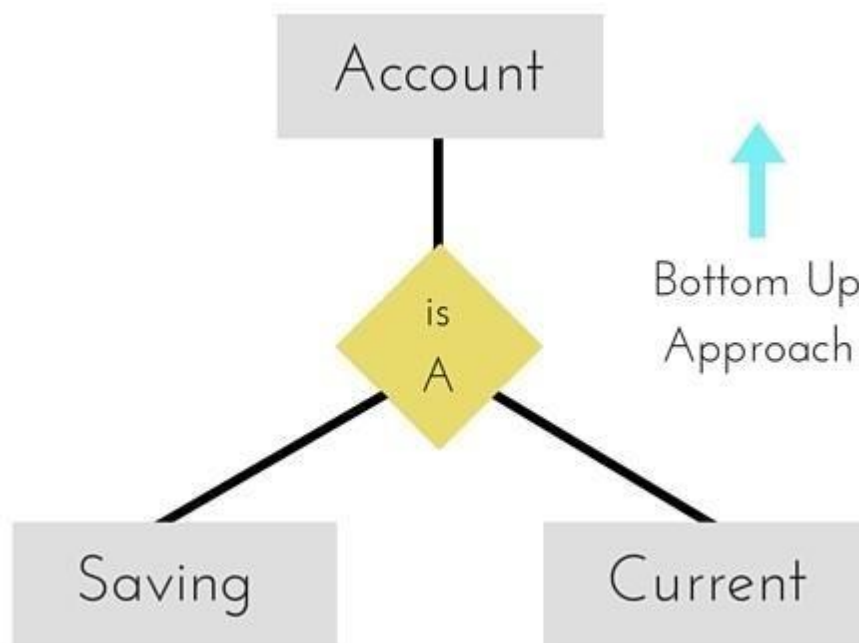


‘Eat’ is common to all animals but ‘roaring’ is a specific to lion and ‘barking’ is specific to dog.

Generalization

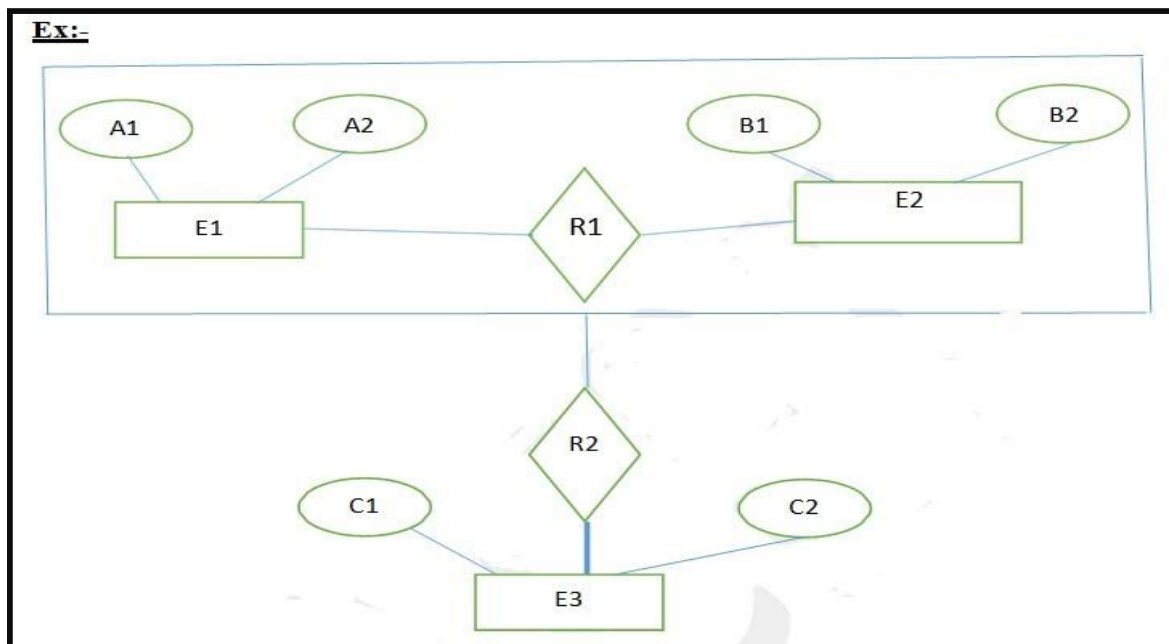
Generalization is a bottom-up approach in which two lower level entities combine to form a higher level entity. In generalization, the higher level entity can also combine with other lower level entities to make further higher level entity.

It's more like Superclass and Subclass system, but the only difference is the approach, which is bottom-up. Hence, entities are combined to form a more generalised entity, in other words, sub-classes are combined to form a super-class.



Aggregation

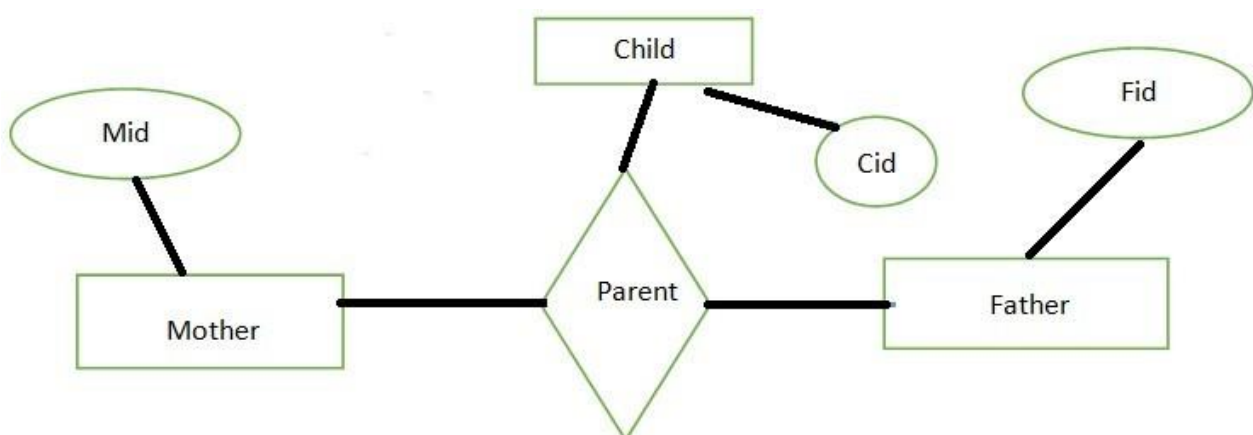
The limitation of E-R model is that, it fails to express relationship among the relationships sets aggregation provides solution for this. In aggregation relationship sets are treated as higher level entity sets which can participate in relationship sets with other entity sets.



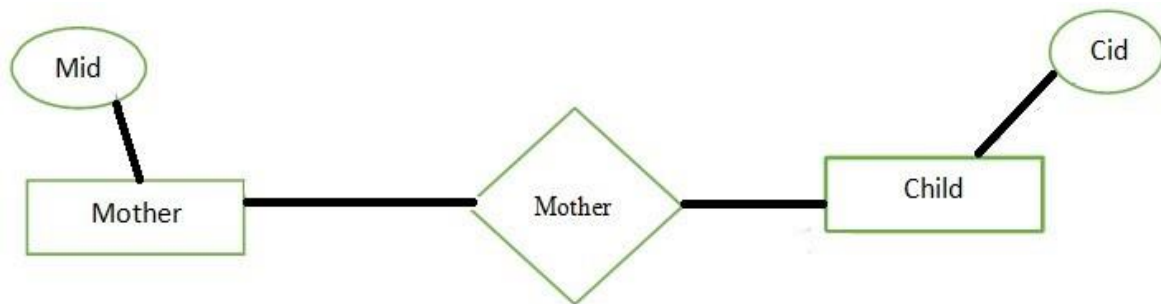
- The relationship **R1** between entity sets **E1** and **E2** has been aggregated as higher-level entity asset “**E**”. This higher-level entity set will participating in a relationship set **R2** with an entity set **E3**.
- So, through aggregation we are able to form a relationship between two relationship sets.

Ternary relationship:

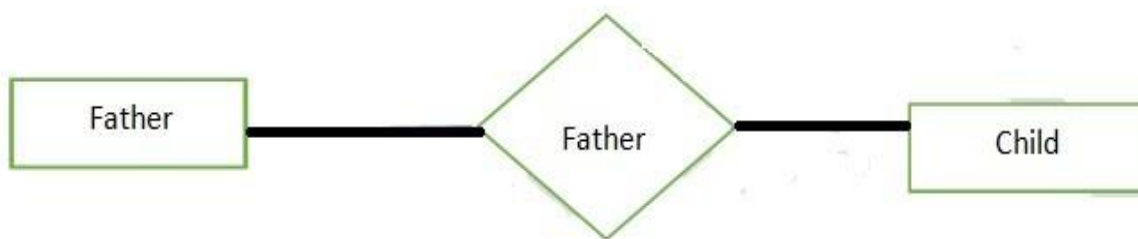
Relationships in database are often binary. But there may be some relationships which are non-binary.



- The above diagram is an example of ternary relationship. The relationship set is a parent is formed by the participation of three entity sets mother, father and child.
- We can divide the above ternary relationship into two binary relationships as shown below.



Mother is the relationship between child and mother entity sets.



Father is the relationship between father and child entity sets.

Using two relationships mother and father provides us a record of a child's mother even if we are not aware of the father's identity, provides us a record of a child's father, even if we are not aware of the mother identity. A null value would be required if the ternary relationship set parent is used in two cases. Using of binary relationship preferable in this case.

Converting of E-R diagrams into relational

An E-R diagram can be reduced to a set of tables while reducing

Points to be remember: -

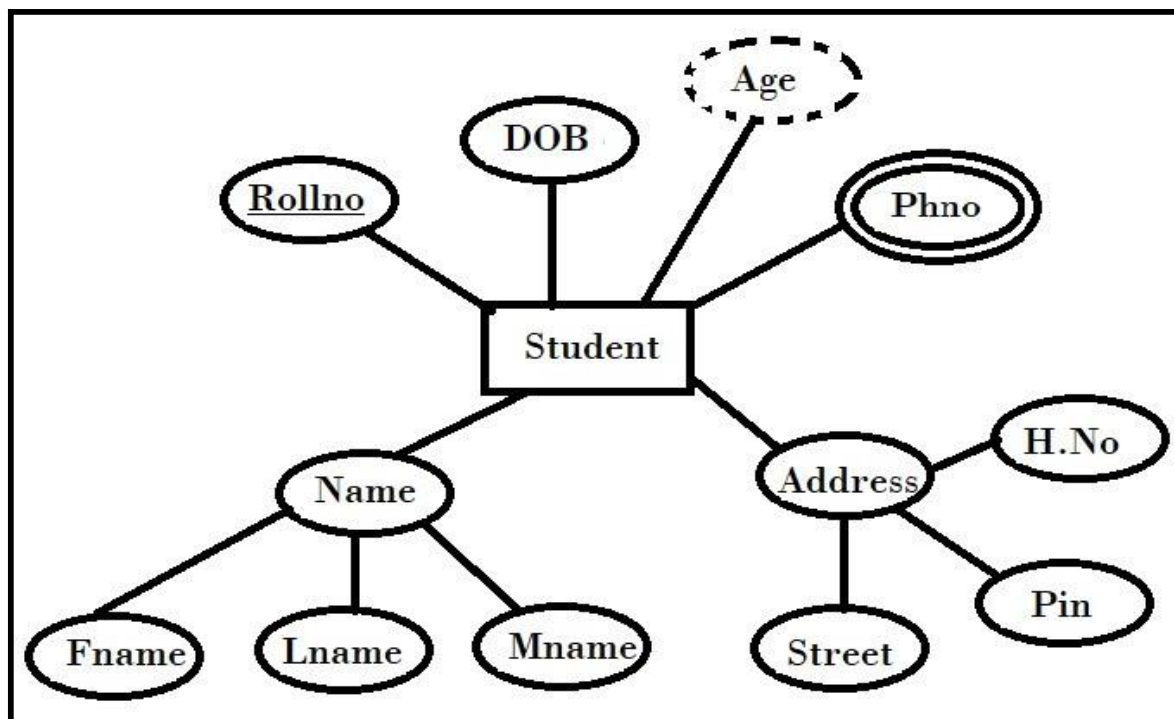
- There will be a separate column for each simple and single valued attributes of given entity set.
- There will be a separate column for each sub-attribute and there is no column for composite attribute.
- There is no separate column for derived attribute
- Each multi valued attribute will be represented in a separate table which will have a column for primary key attribute and multi valued attribute.
- There is a separate table for each entity set and relationship set.

DATABASE MANAGEMENT SYSTEMS

Tabular Representation of Strong Entity Set

Entity in ER Model is changed into tables, or we can say for every Entity in ER model, a table is created in Relational Model.

And the **attributes** of the Entity gets converted to columns of the table. And the primary key specified for the entity in the ER model, will become the primary key for the table in relational model.



A table with name **Student** will be created in relational model, which will have 8 columns Rollno, DOB, Fname, Lname, Mname, Hno, Street, Pin and Rollno will be the primary key for this table.

Rollno	DOB	Fname	Lname	Mname	Hno	Street	Pin
--------	-----	-------	-------	-------	-----	--------	-----

Student

Another Table with a name **Student_Phno** will be created in relational model, which have 2 columns Rollno, Phno.

Rollno	Phno
--------	------

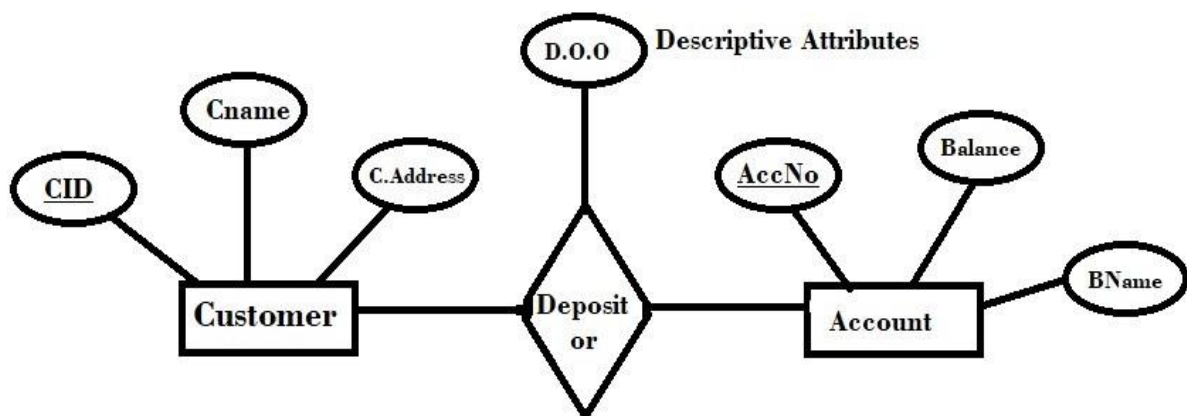
Student_Phno

DATABASE MANAGEMENT SYSTEMS

Tabular Representation of Relationship Set

In ER diagram, we use diamond/rhombus to represent a relationship between two entities. In Relational model we create a relationship table for ER Model relationships too.

In the ER diagram below, we have two entities **Customer** and **Account** with a relationship between them.



There are three tables for Customer, Account and Depositor.

As discussed above, entity gets mapped to table, hence we will create table for **Customer** and a table for **Account** with all the attributes converted into columns.

CID	CName	C.Address
------------	--------------	------------------

Customer

AccNo	Balance	Bname
--------------	----------------	--------------

Account

Now, an additional table will be created for the relationship, for example **Depositor**. This table will hold the primary key for both Customer and Account, in a tuple to describe the

DATABASE MANAGEMENT SYSTEMS

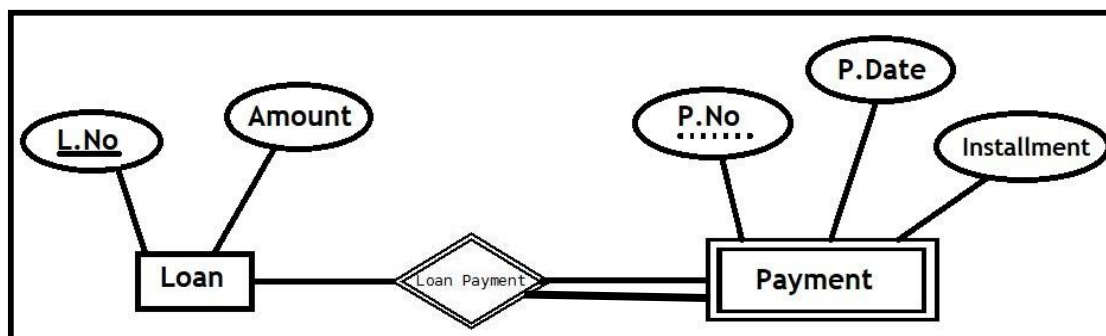
relationship, which Customer Depositor which Account. If there are additional attributes related to this relationship, then they become the columns for this table, like **D.O.O**.

CID	AccNo	D.O.O
------------	--------------	--------------

Depositor

Tabular Representation of Weak Entity Set

In the below ER Diagram, '**Payment**' is the weak entity. 'Loan Payment' is the identifying relationship and 'Payment Number' is the partial key. Primary Key of the Loan along with the partial key would be used to identify the records.



There are three tables for Loan, Payment and Loan_Payment.

L.No	Amount
-------------	---------------

Loan

L.No	P.No	P.Date	Installment
-------------	-------------	---------------	--------------------

Payment

L.No	P.No
-------------	-------------

Loan_Payment

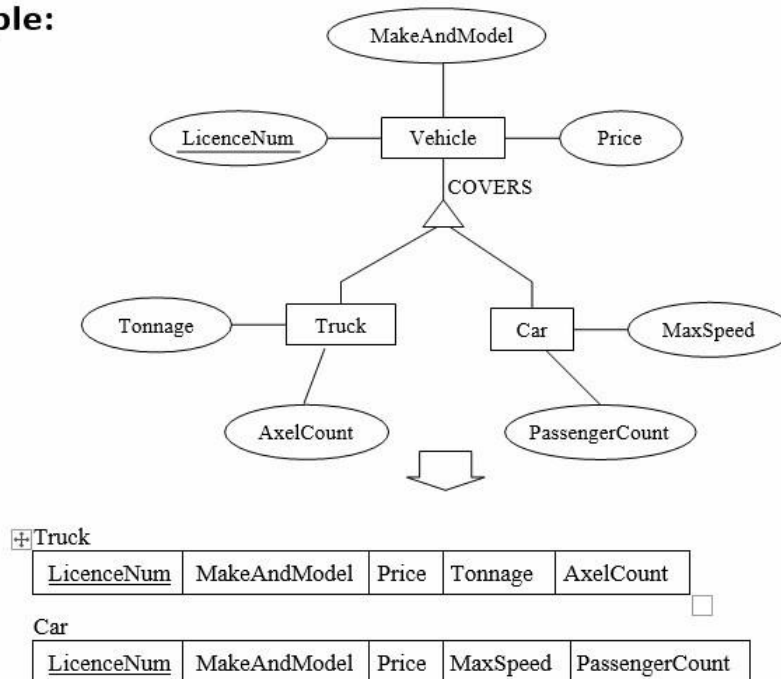
Tabular Representation of Generalization

- Create a table for each lower-level entity set only
- Columns of new tables should include
 - Attributes of lower-level entity set
 - Attributes of the superset

DATABASE MANAGEMENT SYSTEMS

- The higher-level entity set can be defined as a view on the tables for the lower-level entity sets.

Example:



Tabular Representation of Aggregation

To represent relationship set involving aggregation of R, treat the aggregation like an entity set whose primary key is the **primary key** of the table for R.

Example:

